Salient Object Detection from Scratch

End-to-End Machine Learning / Deep Learning Project

Dataset : ECSSD (1,000 images)

Framework : PyTorch

Input Size: 128 x 128 px

Key Results

Baseline IoU = 0.5890 F1 = 0.7176 MAE = 0.1658

Improved IoU = 0.6235 F1 = 0.7436 MAE = 0.1501

IoU improvement: +5.9%

Cohort V | 2026

1. Introduction and Background

Salient Object Detection (SOD) is a computer vision task that identifies and segments the most visually dominant region in an image - the area that naturally draws human attention. Unlike general object detection, which classifies every object present, SOD produces a unified binary saliency map that highlights the most perceptually important region regardless of its category.

Applications of SOD include image cropping, video surveillance, medical image analysis, and image compression. The core challenge is teaching a model to replicate the human visual system's ability to distinguish foreground subjects from complex backgrounds.

This project implements a complete SOD pipeline from scratch - without any pre-trained weights - covering dataset preparation, custom CNN architecture design, training with a combined loss function, quantitative evaluation, and an interactive live demo.

2. Dataset

2.1 Dataset Selection

The ECSSD (Extended Complex Scene Saliency Dataset) was selected. ECSSD contains 1,000 natural images of semantically meaningful, complex scenes, each paired with a pixel-accurate binary saliency mask annotated by human observers. It is a standard SOD benchmark suited for experimentation due to its manageable size and high annotation quality.

2.2 Preprocessing

Step	Details
Loading	Images as RGB; masks as grayscale
Resizing	128 x 128 px (bilinear for images, nearest for masks)
Normalisation	Pixel values scaled to [0, 1]
Binarisation	Mask threshold at 0.5

2.3 Dataset Splits

Split	Images	Percentage
Training	699	69.9%
Validation	150	15.0%
Test	151	15.1%

2.4 Data Augmentation (training set only)

Augmentation	Probability	Parameters
Horizontal flip	50%	-
Vertical flip	50%	-

Random rotation	50%	+-30 degrees
Random crop + resize	50%	crop ratio in [0.70, 1.0]
Brightness jitter	50%	factor in [0.60, 1.40]
Contrast jitter	50%	factor in [0.60, 1.40]
Saturation jitter	50%	factor in [0.50, 1.50]
Gaussian noise	60%	sigma = 0.025
Random erasing	70%	1-3 patches, area 2-20%

3. Model Architecture

3.1 Baseline - BaselineSODNet

The baseline model follows a symmetric encoder-decoder design without skip connections. The encoder progressively downsamples spatial resolution while increasing channel depth. The decoder mirrors this by upsampling back to input resolution. BatchNorm is applied after every convolution for training stability, and Dropout2d(0.3) is applied at the bottleneck to prevent overfitting on the small training set.

Stage	Layer	Channels	Output Size
Encoder 1	Conv3x3 + BN + ReLU + MaxPool	3 -> 64	64 x 64
Encoder 2	Conv3x3 + BN + ReLU + MaxPool	64 -> 128	32 x 32
Encoder 3	Conv3x3 + BN + ReLU + MaxPool	128 -> 256	16 x 16
Encoder 4	Conv3x3 + BN + ReLU + MaxPool	256 -> 512	8 x 8
Bottleneck	Conv3x3 + BN + ReLU + Drop2d	512 -> 512	8 x 8
Decoder 4	ConvTranspose + BN + ReLU	512 -> 256	16 x 16
Decoder 3	ConvTranspose + BN + ReLU	256 -> 128	32 x 32
Decoder 2	ConvTranspose + BN + ReLU	128 -> 64	64 x 64
Decoder 1	ConvTranspose + BN + ReLU	64 -> 32	128 x 128
Head	Conv1x1 + Sigmoid	32 -> 1	128 x 128

Total trainable parameters: 4,609,569

3.2 Improved - ImprovedSODNet

The improved model introduces four targeted architectural changes over the baseline, each addressing a known limitation:

- Skip connections (U-Net style): Encoder feature maps are concatenated with decoder feature maps at each scale, preserving spatial detail lost during pooling.
- ASPP bottleneck (Atrous Spatial Pyramid Pooling): Three parallel dilated convolutions (dilation rates 1, 2, 4) plus a global average-pooling branch capture multi-scale context at the 8x8 bottleneck.
- Attention gates on skip connections: Soft attention weights are computed from the gating signal (decoder) and skip feature (encoder), suppressing irrelevant background features before merging.
- Deeper double-conv encoder blocks: Two ConvBNReLU layers per encoder stage instead of one, increasing the depth of learned representations before each pooling step.

```
Input (3, 128, 128)
  enc1: 2x[Conv+BN+ReLU]  ->  (32, 128, 128)
  enc2: 2x[Conv+BN+ReLU]  ->  (64, 64, 64)
  enc3: 2x[Conv+BN+ReLU]  ->  (128, 32, 32)
  enc4: 2x[Conv+BN+ReLU]  ->  (256, 16, 16)
  ASPP bottleneck         ->  (256, 8, 8)
  up4 + AttGate(e4) + dec ->  (128, 16, 16)
  up3 + AttGate(e3) + dec ->  (64, 32, 32)
  up2 + AttGate(e2) + dec ->  (32, 64, 64)
```

```
  up1 + AttGate(e1) + dec ->  (16, 128, 128)
  Conv1x1 + Sigmoid       ->  (1, 128, 128)
```

Total trainable parameters: 4,264,829

4. Training Configuration

4.1 Loss Function

A three-term combined loss is used to optimise both pixel-level accuracy and structural consistency of the predicted saliency map:

```
L = Focal-BCE(pred, target)  +  1.0 x (1 - IoU)  +  0.5 x (1 - SSIM)
```

- Focal-BCE (gamma=2): Down-weights easy background pixels, forcing the model to focus on hard boundary regions. Addresses the class imbalance common in SOD.
- IoU loss: Directly optimises the evaluation metric, encouraging compact and well-overlapping mask predictions.
- SSIM loss: Enforces structural consistency by comparing local means, variances, and covariances - producing sharper, more coherent mask edges.

4.2 Optimiser and Schedule

Hyperparameter	Value
Optimiser	AdamW (decoupled weight decay)
Learning rate	5 x 10^-4
Weight decay	1 x 10^-4
LR schedule	Linear warmup (3 epochs) + Cosine decay
Gradient clipping	Max norm = 1.0
Max epochs	80
Early stopping	Patience = 15 on val IoU
Batch size	16

4.3 Checkpoint Strategy [Bonus]

Two checkpoints are maintained per training run. The 'last' checkpoint saves model weights, optimiser state, epoch index, and full training history after every epoch, enabling full resume from any interruption. The 'best' checkpoint is saved only when validation IoU improves, and is used for final evaluation.

5. Experiments and Results

5.1 Test Set Evaluation

Both models were evaluated on the held-out test set of 151 images using a prediction threshold of 0.5. All metrics are averaged over the full test set.

Metric	Baseline	Improved	Change
IoU	0.5890	0.6235	+5.9%
Precision	0.6827	0.7269	+6.5%
Recall	0.8363	0.8253	-1.3%
F1-Score	0.7176	0.7436	+3.6%
MAE	0.1658	0.1501	-9.5%

5.2 Training History

Model	Epochs Trained	Best Val IoU	Final Val IoU
Baseline	80	0.5867	0.5863
Improved	80	0.6182	0.6178

5.3 Discussion

The improved model outperforms the baseline on every metric. The most significant gains are in IoU (+5.9%) and MAE (-9.5%), indicating that the architectural changes (ASPP + attention gates + skip connections) produce substantially better-aligned and sharper saliency masks.

The slightly lower Recall in the improved model (-1.3%) is expected: the attention gates and precision-focused loss suppress false positives, making the model more selective. The trade-off results in higher Precision (+6.5%) and a better F1.

An IoU of 0.6235 is strong for a model trained from random initialisation on only 699 images. State-of-the-art SOD models achieve IoU > 0.85, but these universally rely on ImageNet-pretrained encoders and datasets with 10,000+ training images. Training from scratch on ECSSD demonstrates that principled architecture and loss design can still yield meaningful results.

5.4 Improvement Summary

Feature	Baseline	Improved	Rationale
Skip connections	No	Yes	Preserves spatial detail lost at bottleneck
ASPP bottleneck	No	Yes	Multi-scale context at 8x8 feature map
Attention gates	No	Yes	Filters irrelevant skip features
Double conv blocks	No	Yes	Deeper per-stage representations
BatchNorm	Yes	Yes	Stable gradients throughout
Dropout2d (0.3)	Bottleneck only	Bottleneck + decode	Regularisation
Loss: Focal-BCE	Yes	Yes	Handles class imbalance

Loss: IoU term	Yes	Yes	Direct metric optimisation
Loss: SSIM term	Yes	Yes	Structural consistency

6. Demo

An interactive Streamlit web application (app.py) was built to demonstrate the model in real time. The demo allows users to upload any RGB image, adjust the prediction threshold via a slider, and instantly view:

- The input image
- The saliency probability heatmap (HOT colour map)
- The overlay: input image with red tint on predicted salient region
- Inference time per image

The model is loaded once and cached. Inference on CPU takes approximately 15-30 ms per image at 128x128 resolution. A Jupyter notebook (demo_notebook.ipynb) is also provided, designed for Google Colab with GPU support.

```
streamlit run app.py
```

7. Learnings and Reflections

- Skip connections are the single most impactful architectural change for dense prediction tasks. MaxPooling discards significant spatial information; feeding encoder features directly to the decoder is essential for recovering sharp object boundaries.
- The combined Focal-BCE + IoU + SSIM loss produces visually sharper and more structurally coherent masks than BCE alone. Each term targets a different failure mode: class imbalance, overlap quality, and boundary sharpness.
- Dataset size is the primary bottleneck when training from scratch. With only 699 training images, augmentation (9 transforms) and regularisation (dropout, weight decay) are essential. A dataset like DUTS-TR (10,553 images) would likely push IoU to 0.70+.
- AdamW with decoupled weight decay consistently outperforms Adam + L2 regularisation on small datasets. The distinction matters most when the adaptive gradient scaling in Adam would otherwise incorrectly scale the weight decay term.
- Warmup + cosine annealing eliminates the validation IoU oscillation observed with ReduceLROnPlateau. A smooth, predictable LR trajectory is important when training data is scarce and each epoch's gradient estimate is noisy.
- GPU access has a large practical impact: a full 80-epoch run on CPU takes several hours, limiting the number of experiments possible. The code automatically uses CUDA when available. Running on Google Colab T4 reduces training time to under 15 minutes.

8. References

[1] Shi J. et al. (2015). Hierarchical Image Saliency Detection on Extended CSSD. IEEE TPAMI.

[2] Ronneberger O. et al. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. MICCAI.

[3] Chen L.-C. et al. (2018). Encoder-Decoder with Atrous Separable Convolution (DeepLab v3+). ECCV.

[4] Oktay O. et al. (2018). Attention U-Net: Learning Where to Look for the Pancreas. MIDL.

[5] Ioffe S. & Szegedy C. (2015). Batch Normalization: Accelerating Deep Network Training. ICML.

[6] Loshchilov I. & Hutter F. (2019). Decoupled Weight Decay Regularization (AdamW). ICLR.

[7] Lin T.-Y. et al. (2017). Focal Loss for Dense Object Detection. ICCV.

[8] Kingma D. & Ba J. (2015). Adam: A Method for Stochastic Optimization. ICLR.

[9] Wang Z. et al. (2004). Image Quality Assessment: From Error Visibility to SSIM. IEEE TIP.